

PLAN DEVOPS

**TÉCNICO SUPERIOR UNIVERSITARIO
DESARROLLO DE SOFTWARE**

PRESENTA:

BUSTILLO MOLINA PALOMA JIMENA

CORONEL LIMIAS HECTOR

CRUZ VERDE DANIEL MICHEL

PÉREZ BARRIOS ERNESTO

RODRÍGUEZ GÓMEZ MARÍA BELÉN

PROFESOR:

SALDAÑA BENÍTEZ HÉCTOR

SAN JUAN DEL RIO, QRO

SEPTIEMBRE 2026

INDICE

1. RESUMEN DEL CASO Y OBJETIVOS	3
Caso elegido	3
Problema.....	3
Solución.....	3
Usuarios.....	3
Objetivos.....	3
Métricas de éxito	4
2. ARQUITECTURA GENERAL Y DEPENDENCIAS	5
Componentes	5
Comunicación.....	5
3. CICLO DEVOPS APLICADO.....	6
4. VERSIONAMIENTO Y RAMIFICACIÓN.....	7
Convención de commits	7
Versionamiento	7
5. CI/CD.....	8
Integración continua	8
Criterios para realizar un merge	8
Release.....	8
6. ESTRATEGIA DE PRUEBAS	9
Criterios de entrada.....	9
Criterios de salida	9
7. ESTRATEGIA DE DESPLIEGUE.....	10
8. MONITOREO Y MÉTRICAS	11
Aspectos por monitorear.....	11
Indicadores.....	11
9. RIESGOS Y PLANES DE MITIGACIÓN	12
10. GUÍA DE RUNBOOK PARA FALLAS COMUNES	13

1. RESUMEN DEL CASO Y OBJETIVOS

Caso elegido

Reservación de canchas de la universidad

Problema

Actualmente, la organización de la utilización de las canchas de fútbol dentro de la escuela puede resultar complicada porque la información sobre equipos, jugadores, partidos, horarios, resultados y disponibilidad de cancha se maneja mediante diferentes medios como grupos de WhatsApp, hojas de cálculo, formularios o comunicación directa. Esto puede generar pérdida de información, confusión, problemas con los horarios y dificultad para consultar información actualizada.

Solución

FutSchool será una aplicación móvil para gestionar y organizar torneos de fútbol escolares. Permitirá consultar torneos, registrar equipos y jugadores, solicitar inscripciones, gestionar partidos, asignar horarios y canchas, registrar resultados y consultar la tabla de posiciones. También contará con funciones de notificaciones y administración de torneos.

Usuarios

- Estudiante/capitán.
- Administrador.

El estudiante/capitán podrá registrar equipos, administrar jugadores, solicitar inscripciones y consultar partidos, calendario y resultados. El administrador podrá crear torneos, gestionar equipos e inscripciones, registrar canchas, programar partidos, registrar resultados y administrar la información del torneo.

Objetivos

- Centralizar la información de los torneos.
- Facilitar la consulta de partidos, horarios, canchas y resultados.
- Permitir la administración de equipos, jugadores y torneos.
- Mantener actualizada la tabla de posiciones.
- Permitir el flujo completo desde el registro de un equipo hasta el registro de resultados.

Métricas de éxito

1. Registro de usuarios funcional.
2. Flujo completo funcional: registro → equipo → torneo → inscripción → partido → cancha → resultado → tabla.
3. Cumplimiento de los requisitos funcionales y no funcionales definidos para la aplicación.

Estas métricas se basan en los resultados funcionales y criterios de validación establecidos en la planeación de FutSchool.

2. ARQUITECTURA GENERAL Y DEPENDENCIAS

FutSchool utiliza una arquitectura **Cliente-Servidor basada en una API REST**, organizada en las capas de presentación, comunicación, lógica de negocio y datos. La aplicación móvil funciona como cliente y se comunica mediante Internet con la API REST. La API comunica las peticiones con el backend, mientras que el backend procesa la lógica de negocio y consulta o modifica la base de datos.

Componentes

Componente	Función
Aplicación móvil	Interfaz utilizada por estudiantes/capitanes y administradores.
API REST	Recibe las peticiones de la aplicación móvil y las dirige al backend.
Backend	Contiene la lógica de negocio, validaciones y reglas del sistema.
Base de datos	Almacena usuarios, equipos, jugadores, torneos, inscripciones, canchas, partidos, resultados y notificaciones.
Autenticación y roles	Valida credenciales, maneja el token y controla los permisos.
Servicio de notificaciones	Envía avisos sobre próximos partidos y cambios de horario o cancha.

Comunicación

La aplicación móvil envía peticiones HTTP/HTTPS utilizando JSON hacia la API REST. La API comunica las peticiones con el backend, que realiza las validaciones y reglas correspondientes. El backend también se comunica con la base de datos y con el servicio externo de notificaciones cuando ocurre un evento importante.

3. CICLO DEVOPS APLICADO

Fase	Objetivo	Herramienta(s)	Artefactos/Salidas	Criterio de hecho
Plan	Organizar funcionalidades y tareas	GitHub Projects	Backlog, historias	Historias con criterios de aceptación
Code	Desarrollar las funcionalidades	VS code / Flutter	Commits y PRs	PR con 1 reviewer
Build	Comprobar la construcción	GitHub Actions	Artefacto de build	Build sin errores
Test	Comprobar las funcionalidades	Flutter test	Reportes de pruebas	Pruebas ejecutadas
Package	Preparar el paquete de la aplicación	Gradle/Fastlane	APK/AAB/IPA	Artefacto preparado
Release	Preparar una versión	Fastlane/TestFlight	Tag/Release	Tag v0.x.y
Deploy	Publicar la versión	Firebase App Distribution / App Store	Versión publicada	Checklist OK
Operate	Mantener la aplicación funcionando	Crashlytics/Sentry	Dashboards y alertas	Alertas activas
Monitor	Revisar el funcionamiento	Grafana/Console	KPIs/SLIs	Revisión semanal
Feedback	Recibir y registrar mejoras	Issues/Encuestas	Retros y acciones	Acciones registradas

4. VERSIONAMIENTO Y RAMIFICACIÓN

Se utilizará una estrategia de ramas basada en GitHub Flow, manteniendo main como la rama principal y estable del proyecto. El desarrollo de nuevas funcionalidades se realizará mediante ramas independientes.

main: rama principal y estable.

Develop/*: ramas utilizadas para desarrollar nuevas funcionalidades.

Cuando una funcionalidad esté terminada, se realizará un Pull Request hacia main para revisar e integrar los cambios. De esta manera, el proyecto podrá avanzar por funcionalidades sin modificar directamente la rama principal.

Convención de commits

Se utilizará Conventional Commits para identificar de manera sencilla el tipo de cambio realizado.

Ejemplos:

- feat: agregar registro de equipos
- fix: corregir validación de partidos
- docs: actualizar documentación

Versionamiento

Se utilizará **Semantic Versioning**, comenzando con una versión de prototipo como:

v0.1.0

La práctica establece el uso de tagging semántico y utiliza v0.1.0 como ejemplo para el prototipo.

5. CI/CD

Integración continua

El repositorio utilizará GitHub Actions para ejecutar un workflow de integración continua. Siguiendo GitHub Flow, main será la rama principal y se crearán ramas temporales según el trabajo que se realice; no se requiere una rama fija adicional.

El workflow se ejecutará cuando se abra o actualice un pull request dirigido a main y cuando se incorporen cambios a main.

En la etapa actual, como el repositorio todavía contiene documentación y no hay código de la aplicación, el workflow comprobará que exista el archivo README.md. Cuando se incorpore la aplicación, esta comprobación se ampliará para ejecutar un build o una prueba rápida de lint o test.

El repositorio contará con una plantilla de issue para describir tareas o problemas y una plantilla de pull request para indicar los cambios realizados y cómo se revisaron.

El objetivo mínimo será realizar el build o un lint/test rápido de la aplicación.

Criterios para realizar un merge

- Los cambios deberán realizarse en una rama creada para la tarea correspondiente y presentarse mediante un pull request hacia main.
- El pull request deberá pasar los checks de CI disponibles en esa etapa del proyecto.
- El pull request deberá contar con la revisión de otro integrante.
- Los cambios deberán estar relacionados con la tarea descrita en el pull request.

Release

Las versiones utilizarán tags semánticos como v0.1.0. La práctica contempla Fastlane/TestFlight para Release y Firebase App Distribution o las tiendas correspondientes para Deploy.

6. ESTRATEGIA DE PRUEBAS

Las pruebas de FutSchool se realizarán de manera progresiva durante el desarrollo. Cada funcionalidad será revisada una vez implementada para comprobar que cumpla con el resultado esperado. Si se encuentra algún error, se corregirá antes de continuar con funcionalidades que dependan de ella.

También se revisará que los cambios realizados no afecten las funcionalidades que ya se encuentran funcionando.

Criterios de entrada

- La funcionalidad se encuentra implementada.
- La historia de usuario y sus criterios de aceptación están definidos.
- La funcionalidad puede ser ejecutada para su revisión.

Criterios de salida

- La funcionalidad cumple con el resultado esperado.
- Los errores encontrados fueron corregidos.
- Las funcionalidades relacionadas continúan funcionando correctamente.

7. ESTRATEGIA DE DESPLIEGUE

El despliegue de FutSchool se realizará conforme avance el desarrollo del proyecto. Primero se contará con versiones de prueba para comprobar el funcionamiento de las funcionalidades implementadas. Posteriormente, cuando la aplicación se encuentre preparada, se podrá realizar su publicación en las plataformas correspondientes.

El proceso será:

1. Generar una versión de la aplicación.
2. Revisar que las funcionalidades principales funcionen correctamente.
3. Distribuir la versión de prueba para su revisión.
4. Corregir los errores encontrados.
5. Preparar la versión correspondiente para su publicación.

Para las versiones de prueba se contempla el uso de **Firebase App Distribution**, mientras que para una versión de producción se contempla su publicación mediante las tiendas correspondientes.

8. MONITOREO Y MÉTRICAS

El monitoreo se enfocará en los problemas y funcionalidades identificadas en FutSchool.

Aspectos por monitorear

- Errores o fallas de la aplicación.
- Funcionamiento de las funcionalidades principales.
- Cambios de horarios y canchas.
- Registro de resultados.
- Actualización de la tabla de posiciones.
- Notificaciones.

La práctica propone utilizar Crashlytics/Sentry para Operate y Grafana/Console para Monitor.

Indicadores

- Registro de usuarios funcionando.
- Partidos correctamente programados.
- Resultados correctamente registrados.
- Tabla de posiciones actualizada.
- Flujo completo de la aplicación funcionando.

9. RIESGOS Y PLANES DE MITIGACIÓN

Riesgo	Mitigación
Pérdida o desactualización de información	Centralizar la información en la aplicación y base de datos.
Conflictos de horarios y canchas	Revisar la disponibilidad antes de asignar horarios y canchas.
Acceso incorrecto a funciones	Utilizar autenticación y permisos de acuerdo con el tipo de usuario.
Acoplamiento entre partidos y notificaciones	Utilizar el patrón Observer para reducir el acoplamiento.
Errores durante el desarrollo	Realizar pruebas progresivas y corregir errores prioritarios.
Los riesgos serán revisados durante el avance del proyecto. Cuando se detecte un problema, se atenderá de acuerdo con su importancia para evitar que genere retrasos en las siguientes funcionalidades.	

10. GUÍA DE RUNBOOK PARA FALLAS COMUNES

El Runbook se establecieron acciones sencillas para atender problemas que puedan aparecer durante el desarrollo de FutSchool. Cuando se detecte una falla, se identificará el problema, se realizará la corrección y posteriormente se comprobará que vuelva a funcionar correctamente.

Problema	Acción
Build con errores	Revisar el error generado, identificar el cambio que lo ocasionó, corregirlo y volver a ejecutar el proceso.
Credenciales expiradas	Revisar las credenciales utilizadas por el servicio correspondiente y actualizarlas.
Cuota de API alcanzada	Revisar el uso del servicio y ajustar las solicitudes realizadas.
Funcionalidad con errores	Utilizar el patrón Observer para reducir el acoplamiento.
Errores durante el desarrollo	Realizar pruebas progresivas y corregir errores prioritarios.